

CS 3370 - Programming Assignment 6 – Pipelined eCommerce Order Processing System

Prof Flyr

April 6, 2026

A Requirements

Implement a concurrent pipeline for e-commerce.

Due: see Canvas Page

- Implement the assignment described below.
- Write a reflection on what you learned, what you struggled with.
- Turn in onlinegdb.com link with the file(s) including The 5 regional output files (`region_0.txt` through `region_4.txt`)
- See the rubric at the end for more details.
- (Optional) The `pipeline.log` file if you implement shared logging

B Program 6

B-1 Concurrent Order Processing Pipeline

B-1-1 Overview

Write a program with a concurrent pipeline architecture that simulates an e-commerce order processing system with three stages: 1) threads that generate customer orders, 2) threads that validate and filter orders based on business rules, and 3) threads that route validated orders to regional fulfillment centers. This system will use C++20's `jthread` for automatic cleanup and modern exception handling practices. Place all functionality in a class named `OrderPipeline`, ensuring that items related to thread communication are static members. This architecture mirrors real-world distributed order processing systems used by major e-commerce platforms.

B-1-2 System Architecture

The pipeline consists of three stages, as shown in Figure 1:

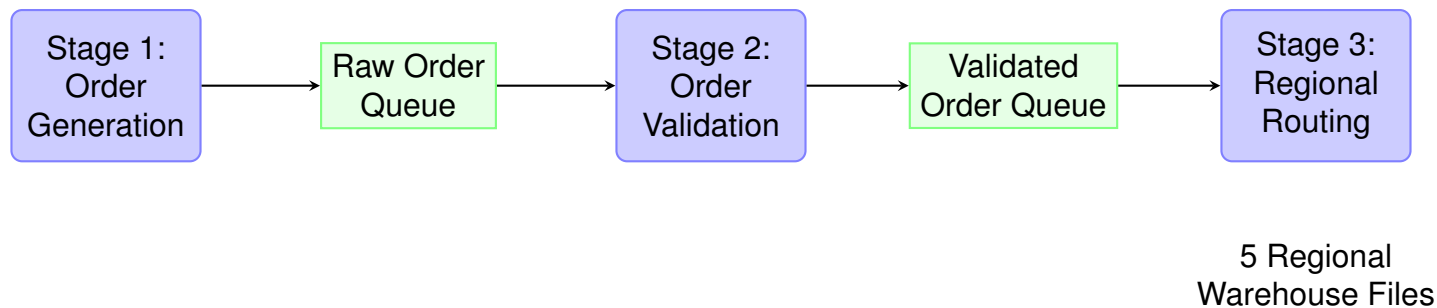


Figure 1: Order Processing Pipeline Architecture

B-1-3 Detailed Requirements

Stage 1: Order Generation (5 Producer Threads) Create 5 producer threads using `jthread` that generate a total of 1000 random customer orders. Each order should contain:

- **Order ID:** A unique integer identifier
- **Order Amount:** A random dollar amount between \$10 and \$500
- **Region Code:** A random integer from 0-4 representing five different regions (West, Central, East, North, South)

These threads push order objects into the first shared queue. Use proper mutex protection and condition variables to coordinate access to this queue.

Stage 2: Order Validation (4 Filter Threads) Create 4 filter threads that validate orders from the first queue. A valid order must satisfy:

- Order amount $\geq$ \$50 (company minimum order policy)
- Order ID must be positive

Invalid orders are discarded. Valid orders are pushed to the second queue. These threads must handle potential exceptions from queue operations gracefully.

Stage 3: Regional Routing (5 Router Threads) Create exactly 5 router threads, one for each region (0-4). Each router thread:

- Monitors the validated order queue
- **Critical Requirement:** Only removes orders from the front of the queue if the order's region code matches the thread's assigned region
- If the order at the front doesn't match, the thread **must leave it in the queue** for the appropriate router thread

- Writes matching orders to its regional warehouse file (`region_0.txt` through `region_4.txt`)
- Each line in the output file should be: `OrderID,Amount`

Important: This selective dequeuing behavior is essential for the system to work correctly. Each router thread must check the front element without removing it, and only proceed if it matches its region.

B-1-4 Technical Requirements

1. **Use C++20 jthread:** All threads must be `std::jthread` instances, which automatically join on destruction and support stop tokens for cooperative cancellation.
2. **Stop Token Usage:** Router threads should check their stop token periodically to enable clean shut-down:

```
void routerThread(std::stop_token token, int region_id) {  
    while (!token.stop_requested()) {  
        // Check queue and process...  
    }  
}
```

3. **Exception Handling:** All file I/O operations must be wrapped in try-catch blocks. If a router thread cannot open its output file, it should:
 - Log an error message to `stderr`
 - Continue attempting to process orders (storing them in memory)
 - Attempt to write accumulated orders periodically
4. **Shared Logging (Optional 10% Bonus):** Implement a thread-safe logging mechanism where each thread can log significant events (order generated, order validated, order routed) to a single `pipeline.log` file. This requires careful synchronization to prevent interleaved log entries. I recommend that you do this last if you choose to implement it.
5. **Synchronization Primitives:**
 - Use `std::mutex` and `std::condition_variable` for queue access

- Distinguish between `notify_one()` and `notify_all()` appropriately
- Use lambda predicates with `.wait()` for condition variables
- Ensure no mutex is held longer than necessary

6. Clean Code Practices (per the rubric):

- No global variables; use static class members only
- Properly initialize all static data members
- Prefer range-based for loops when iterating over thread collections
- DRY code

B-1-5 *Expected Output*

When execution completes, your program should print a report like the following (your exact numbers will vary):

```
Order Processing Pipeline Results
=====
Region 0 (West): 142 orders, Total Revenue: $18,234.50
Region 1 (Central): 138 orders, Total Revenue: $17,891.23
Region 2 (East): 145 orders, Total Revenue: $19,102.78
Region 3 (North): 136 orders, Total Revenue: $17,456.90
Region 4 (South): 141 orders, Total Revenue: $18,678.34

Total Valid Orders: 702
Total Invalid Orders: 298
Pipeline processing completed successfully.
```

Note: The order in which regions report may vary since threads terminate independently. Your total valid orders should be approximately 700 (about 70% pass validation), but exact numbers will vary due to randomness.

B-1-6 Testing Considerations

Your solution should handle these edge cases:

- Producer threads finishing at different times
- All queues becoming empty before all threads complete
- Proper cleanup when using stop tokens
- Exception recovery for file I/O failures
- No deadlock or race conditions

B-1-7 Inclusions

As an FYI, below is a list of inclusions you will probably need if you do the entire assignment with the optional/bonus. Do note that if you do not use all of these, that's ok if the assignment is complete. In particular, `atomics` may be replaced with mutexes if that's easier to comprehend.

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <queue>
#include <fstream>
#include <random>
#include <atomic>
#include <vector>
#include <iomanip>
#include <chrono>
#include <stdexcept>
```

B-1-8 Hints

- The router threads need to **peek** at the front of the queue without removing the element if it doesn't match their region. Consider using `queue.front()` to check before calling `queue.pop()`.
- You'll need a mechanism to signal when producers have finished so validators know to stop, and when validators have finished so routers know to stop. Stop tokens are ideal for this.
- To avoid busy-waiting, router threads should use condition variables to sleep when the queue is empty or when the front element doesn't match their region.
- Keep track of statistics (order counts, revenue totals) in thread-safe counters using atomic operations or mutex-protected variables.
- *wait5.cpp* is uploaded as it has some examples that may be helpful for this assignment (although it does not use `jthreads`). *semaphore.cpp* shows `jthread` usage but does not otherwise necessarily shed light on other aspects of the assignment. (There is no requirement to use semaphores.)

B-2 Assessment Rubric

Competency	Proficient	Exemplary
Concurrency	Effective use of mutexes and condition variables to synchronize threads. No mutex held longer than necessary. Proper use of <code>notify_one()</code> vs <code>notify_all()</code> . Lambdas used with <code>.wait()</code> .	Implements stop token checking correctly. Handles all race conditions. Uses <code>jthread</code> features properly including automatic joining.
Exception Handling	All file operations protected with try-catch blocks. Errors logged appropriately.	Graceful recovery from exceptions. System continues operating even when individual file operations fail.
Clean Code	No repeated code. No global data—uses static class members. Proper initialization of static data. Range-based for loops for thread collections.	Well-organized code structure. Clear separation of concerns. Good use of RAII principles.
Correctness	Regional routing works correctly—each thread only processes its assigned region. Correct data in all 5 output files. Final counts are accurate.	Handles edge cases properly. Clean shutdown with no resource leaks. Optional logging implemented correctly.